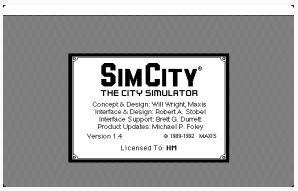
SimCity Tile Diagrams

Chaim Gingold

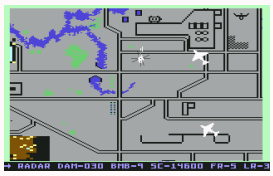
WIP for Dissertation research
Thu Sep 12 2013

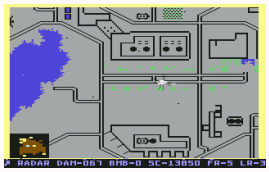
SimCity Versions

Of course there are point releases and so on, but we're not concerned with that.













1984 — Raid on Bungeling Bay C64 — 6502

Will Wright: Design/code

Already has a mini-map

1987? — SimCity C64 Prototype — 6502 — ?

1989 — SimCity C64 — 6502 — ? — SimCity Mac — C and Asm 68000

Will Wright: Design/code

Will Wright (?) (Concept & Design)
Robert A. Stobel (v.1.4 credits: Interface & Design)
Bret G. Durrett (ConnecTile, Map Graphics, …) (Interface Support)
Steve Hales (sound code), 7.20.89 – 5.20.91

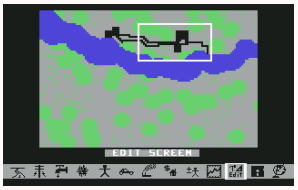
? — SimCity Amiga — C and Asm? 68000 — ? — SimCity PC — C and Asm 8086

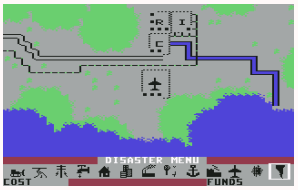
Daniel Goldman: Port
1.6.88 – 6.14.90
See bugs.doc, log.doc in SimCity/DOS/Doc

Other platforms...

1991 — SimCity UNIX/??? — SUN? SPARC? C

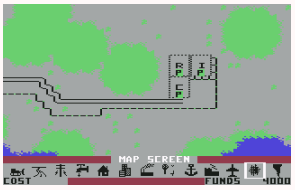
Don Hopkins and Mark Holtz: Port
9.13.91 – 10.27.91? Mark Holtz (MPH) (Immediate Media Company)
12.23.91 – 2.1.92 Don Hopkins (for DUX)
10.27.91 (MPH), see g_power.c?



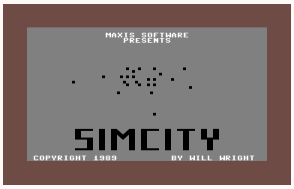


Water network?!!

Note landscape similarity, and roads, to Raid on Bungeling Bay.



Note splatting of tree formations and river from landscape generation. Note that river gets less wide(!?)



? — SimCity OLPC — C, Python? — ? — Micropolis Core OLPC — C++, … ?

Use Wikipedia SimCity entry citations and Archive.org to find missing EA and other documents. (or will wright fan page might have them).

Cellular Automata

Ulam and von Neumann, A.W. Burks
Turing
Toffoli and Margolus (very playful)
Wolfram
Fredkin
Schelling
Conway (popularized; others are useful for understanding).

Games (Computer, Video, ...)

Pong

Graphical User Interface

aka Man Machine Symbiosis
Licklider: NLS, GRAIL
SketchPad
Whirlwind
Xerox PARC
Apple

ASCA flight simulator — Airplane Stability and Control Analyzer

1951 Whirlwind

1962 (?) ARPA IPTO J.C.R. Licklider

1957 DEC

1968 NLS

1966 Ralph Baer Pong

1963 SketchPad — Ivan Sutherland TX-2

1970 Conway's Game of Life

1972 Atari Pong

-1969 GRAIL Rand Corp. — "An Experiment in Man-Machine Communications"

1969- SmallTalk GUI at Xerox PARC

1976 Atari Breakout — Wozniak prototype

Alto Xerox PARC

Urban Simulation

System Dynamics

1977 BASIC Breakout Apple II

Xerox Star

Steve Wozniak: Apple II hardware and software design

1961 Industrial Dynamics

1969 Urban Dynamics

1981 Raster Blaster Apple II

1978 Protoype Apple LISA and Macintosh — LISA released 1983 Mac released 1984

Model Building

1982 Pinball Construction Set Apple II

Games

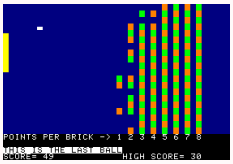
1984 Raid on Bungeling Bay C64

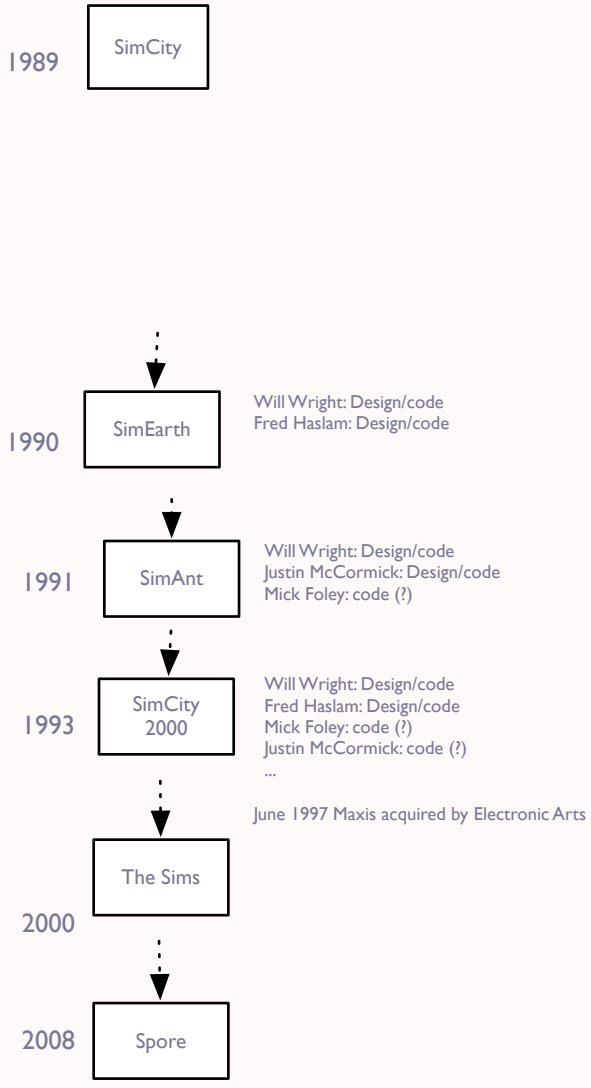
1987? Micropolis C64 Prototype

1989 SimCity

Construction Play

Stanislaw Lem (eg Seventh Sally)
Erector Sets (eg Budge)
Model building kits (e.g. Wright)




1989
SimCity
1990
SimEarth
Will Wright: Design/code
Fred Haslam: Design/code
1991
SimAnt
Will Wright: Design/code
Justin McCormick: Design/code
Mick Foley: code (?)
1993
SimCity 2000
Will Wright: Design/code
Fred Haslam: Design/code
Mick Foley: code (?)
Justin McCormick: code (?)
...
June 1997 Maxis acquired by Electronic Arts
2000
The Sims
2008
Spore

```
extern short *ResHis, ResHisMax;
extern short *ComHis, ComHisMax;
extern short *IndHis, IndHisMax;
extern short *MoneyHis, *CrimeHis, *PollutionHis, *MiscHis;
```

```
/* Graph Histories */
#define RES_HIST          0
#define COM_HIST          1
#define IND_HIST          2
#define MONEY_HIST        3
#define CRIME_HIST        4
#define POLLUTION_HIST          5
#define HISTORIES         6
#define ALL_HISTORIES   ((1 <<HISTORIES) - 1)
```

```
extern unsigned char *History10[HISTORIES], *History120[HISTORIES];
```

```
extern short ResCap, ComCap, IndCap;
extern short RValve, CValve, IValve;



extern short RoadTotal, RailTotal, FirePop;
extern short ResPop, ComPop, IndPop, TotalPop, LastTotalPop;
extern short ResZPop, ComZPop, IndZPop, TotalZPop;
extern short HospPop, ChurchPop, StadiumPop;
extern short PolicePop, FireStPop;
extern short CoalPop, NuclearPop, PortPop, APortPop;
extern short NeedHosp, NeedChurch;
extern short CrimeAverage, PolluteAverage, LVAverage;







extern float roadPercent, policePercent, firePercent;
extern QUAD RoadSpend, PoliceSpend, FireSpend;
extern QUAD roadMaxValue, policeMaxValue, fireMaxValue;
extern QUAD TaxFund, RoadFund, PoliceFund, FireFund;
extern short RoadEffect, PoliceEffect, FireEffect;
extern short TaxFlag, CityTax;
extern QUAD TotalFunds;









 extern short GameLevel;
 extern short Cycle;
 extern short ScenarioID;
 extern short ShakeNow;
 extern QUAD DonDither;
 extern int DoOverlay;



 extern QUAD costOf[];
```

```
extern short CityScore;
extern short deltaCityScore;                extern short DisasterEvent;
extern short ScoreType;                     extern short DisasterWait;
extern short ScoreWait;
extern short CityClass;
extern short PolMaxX, PolMaxY;
extern int PowerStackNum;
extern short TrafficAverage;
extern short PosStackN;
extern short SMapXStack[], SMapYStack[];
extern short Zsource;
extern short HaveLastMessage;
extern short PdestX, PdestY;
extern short CdestX, CdestY;
extern int absDist;
extern short CopFltCnt;

// Godzilla
extern short GodCnt;
extern short GdestX, GdestY;
extern short GorgX, GorgY;
extern short GodControl;

// Copter
extern short CopControl;
extern short CdestX, CdestY;

// Maxima
extern short TrafMaxX, TrafMaxY;
extern short CrimeMaxX, CrimeMaxY;

//
extern short FloodX, FloodY;
extern short CrashX, CrashY;
extern short CCx, CCy;
extern QUAD CityPop, deltaCityPop;
extern char *cityClassStr[6];
extern short CityYes, CityNo;

// Problems
extern short ProblemTable[PROBNUM];
extern short ProblemVotes[PROBNUM];
extern short ProblemOrder[4];

extern QUAD CityAssValue;
```

```
/* Object & Sound Numbers */

#define TRA     1
#define COP     2
#define AIR     3
#define SHI     4
#define GOD     5
#define TOR     6
#define EXP     7
#define BUS     8

/* Max # of Objects */

#define OBJN        9
```

```
extern short NewMap;
extern short NewMapFlags[NMAPS];
extern short NewGraph;
extern short ValveFlag;    // I think this just posts a graphics update
extern short MustUpdateFunds;
extern short MustUpdateOptions;
extern short CensusChanged;
extern short EvalChanged;
extern short MeltX, MeltY;
extern int NeedRest;
extern int ExitReturn;
```

```
extern short SimSpeed;
extern short SimMetaSpeed;
extern short NoDisasters;
extern short autoBulldoze;
extern short autoBudget;
extern short autoGo;
extern short UserSoundOn;

extern short PwrdZCnt;
extern short unPwrdZCnt;
```

```
extern short CityScore;
extern short deltaCityScore;
extern short ScoreType;
extern short ScoreWait;
extern short CityClass;
extern short PolMaxX, PolMaxY;
extern int PowerStackNum;
extern short TrafficAverage;
extern short PosStackN;
extern short SMapXStack[], SMapYStack[];
extern short Zsource;
extern short HaveLastMessage;
extern short PdestX, PdestY;
extern short CdestX, CdestY;
extern int absDist;
extern short CopFltCnt;
extern short GodCnt;
extern short GdestX, GdestY;
extern short GorgX, GorgY;
extern short GodControl;
extern short CopControl;
extern short CdestX, CdestY;
extern short TrafMaxX, TrafMaxY;
extern short CrimeMaxX, CrimeMaxY;
extern short FloodX, FloodY;
extern short CrashX, CrashY;
extern short CCx, CCy;
extern QUAD CityPop, deltaCityPop;
extern char *cityClassStr[6];
extern short CityYes, CityNo;
extern short ProblemTable[PROBNUM];
extern short ProblemVotes[PROBNUM];
extern short ProblemOrder[4];
extern QUAD CityAssValue;

extern short ResCap, ComCap, IndCap;
extern short RValve, CValve, IValve;
extern short PwrdZCnt;
extern short unPwrdZCnt;

extern short SimSpeed;
extern short SimMetaSpeed;
extern short NoDisasters;
extern short autoBulldoze;
extern short autoBudget;
extern short autoGo;
extern short UserSoundOn;

extern short DisasterEvent;
extern short DisasterWait;
```

```
/* comefrom: Simulate */
SendMessages(void)
{
  register z;
  short PowerPop;
  float TM;

  if ((ScenarioID) && (ScoreType) && (ScoreWait)) {
    ScoreWait--;
    if (!ScoreWait)
      DoScenarioScore(ScoreType);
  }

  CheckGrowth();

  TotalZPop = ResZPop + ComZPop + IndZPop;
  PowerPop = NuclearPop + CoalPop;

  z = CityTime & 63;

  switch(z) {

  case 1:
    if ((TotalZPop >>2) >= ResZPop)
      SendMes(1); /* need Res */
    break;
  case 5:
    if ((TotalZPop >>3) >= ComZPop)
      SendMes(2); /* need Com */
    break;
  case 10:
    if ((TotalZPop >>3) >= IndZPop)
      SendMes(3); /* need Ind */
    break;
  case 14:
    if ((TotalZPop > 10) && ((TotalZPop <<1) > RoadTotal))
      SendMes(4);
    break;
  case 18:
    if ((TotalZPop > 50) && (TotalZPop > RailTotal))
      SendMes(5);
    break;
  case 22:
    if ((TotalZPop > 10) && (PowerPop == 0))
      SendMes(6); /* need Power */
    break;
  case 26:
    if ((ResPop > 500) && (StadiumPop == 0)) {
      SendMes(7); /* need Stad */
      ResCap = 1;
    }
    else
      ResCap = 0;
    break;
  case 28:
    if ((IndPop > 70) && (PortPop == 0)) {
      SendMes(8);
      IndCap = 1;
    }
    else IndCap = 0;
    break;
  case 30:
    if ((ComPop > 100) && (APortPop == 0)) {
      SendMes(9);
      ComCap = 1;
    }
    else ComCap = 0;
    break;
  case 32:
    TM = unPwrdZCnt + PwrdZCnt;     /* dec score for unpowered zones */
    if (TM)
      if ((PwrdZCnt / TM) < .7)
            SendMes(15);
    break;
  case 35:
    if (PolluteAverage > /* 80 */ 60)
      SendMes(-10);
    break;
  case 42:
    if (CrimeAverage > 100)
```

Anatomy of a Map[] cell



960 tiles

Zone?
Animate?
Bulldozable?
Burnable?
Conduit?
Powered?

16-bit map cell

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15		
	Character										Status							
LOMASK	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	1023	The low 10 bits (2^10=1024) address the 960 element character library.
ALLBITS	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	64512	The high 6 bits represent additional state information—such as whether the tile is a conduit for power.
ZONEBIT	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1024	The Zone bit indicates whether this is the center—or locus of activity—of a map object.
ANIMBIT	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	2048	?
BULLBIT	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	4096	?
BURNBIT	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	8192	Is this tile burnable?
CONDBIT	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	16384	Does this tile conduct electricity? Both buildings and electrical wires are conductive.
PWRBIT	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	32768	Does this tile have power?
	CChr9																	Map[SMapX,SMapY] is stored in CChr9 and CChr
	CChr																	

Map[] is a 120x100 array of 16-bit map cells.

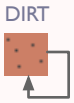
```
#define BLBNBIT     (BULLBIT+BURNBIT)
#define BLBNCNBIT   (BULLBIT+BURNBIT+CONDBIT)
#define BNCNBIT           (BURNBIT+CONDBIT)
```

Tile Animation

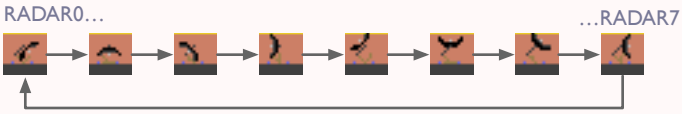
Transitions

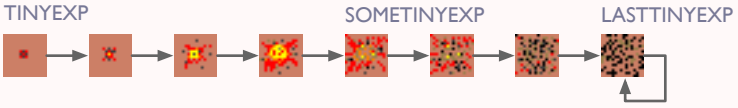
An animation process runs independent of the simulation logic. The simulator simply toggles a tile's ANIMBIT to specify whether an animation should play. animateTiles() scans the entire map, and updates tiles with the ANIMBIT status bit set to on.

A lookup table (aniTile[]) specifies the next tile in the animation sequence for every map tile.

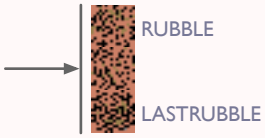


Most tiles simply point to themselves, which makes the system robust in the face of errors in which ANIMBIT might inadvertently be on, and also helps programmers interpret the data table (which is 1024 element array hard coded in C).



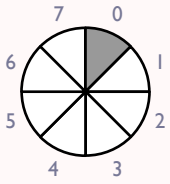


A parallel process in the simulator converts the last four explosion tiles to a non-animating, random landscape RUBBLE tile.



Synchronization

An 8-state clock (tileSynch) controls the phase (synchronization) of animations.

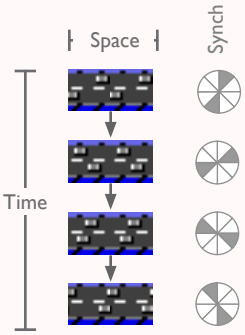


A bit travels around the tileSynch clock. This is bitwise AND'd with the tile's synch entry (aniSynch[]) to test whether the tile should be updated this frame.

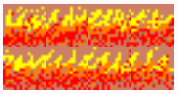




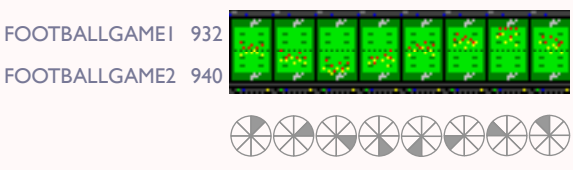
Adjacent traffic tiles whose animation are out of phase will look broken.



Animations are updated every time animateTiles() is called—once their synch bits have guided them into a globally synchronized pattern.

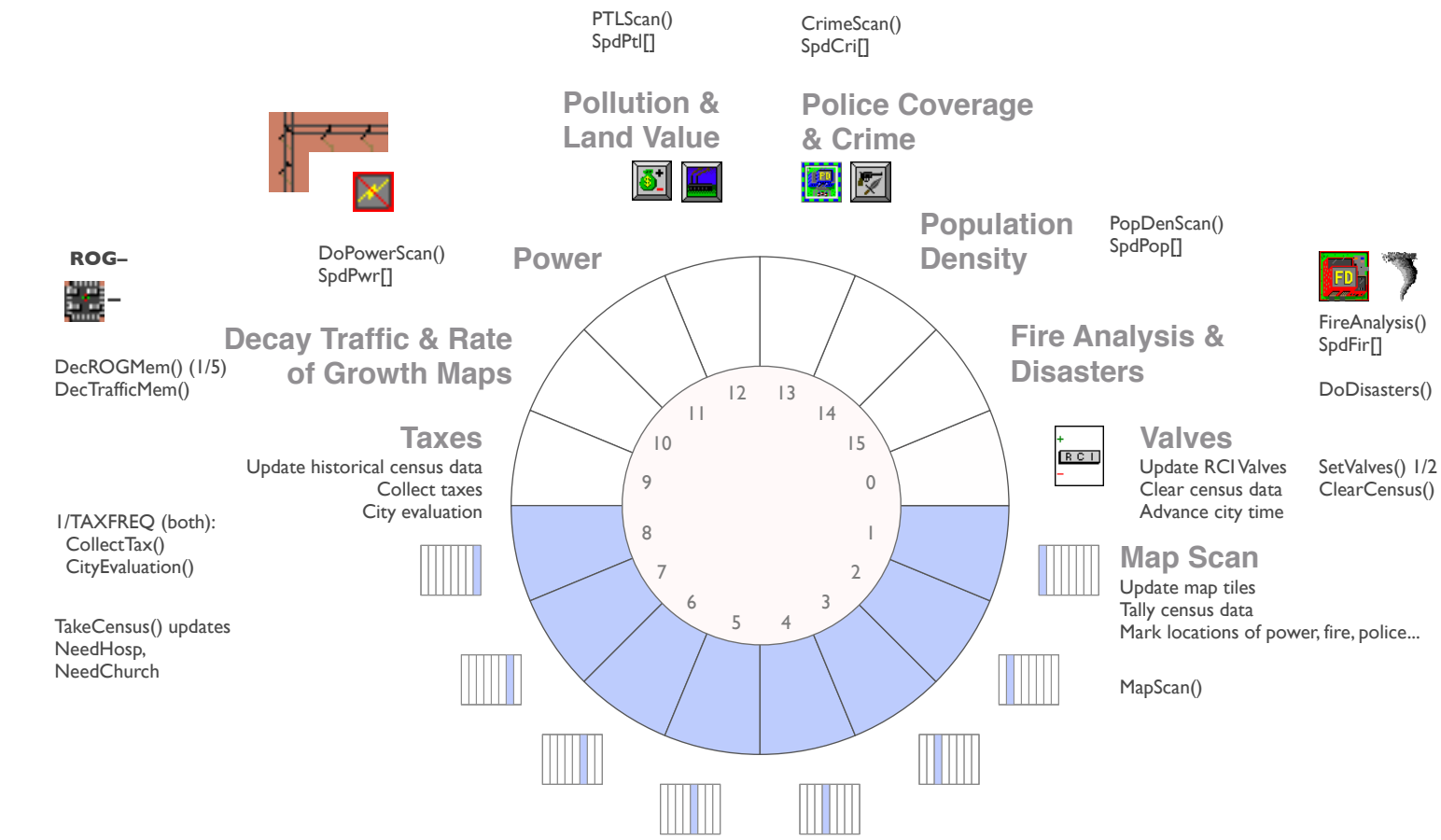


Animations that don't need to be sync'd (e.g. multiple fires) can advance on any frame.



With this technique, animations up to 8 frames long can be synchronized.

Simulate()

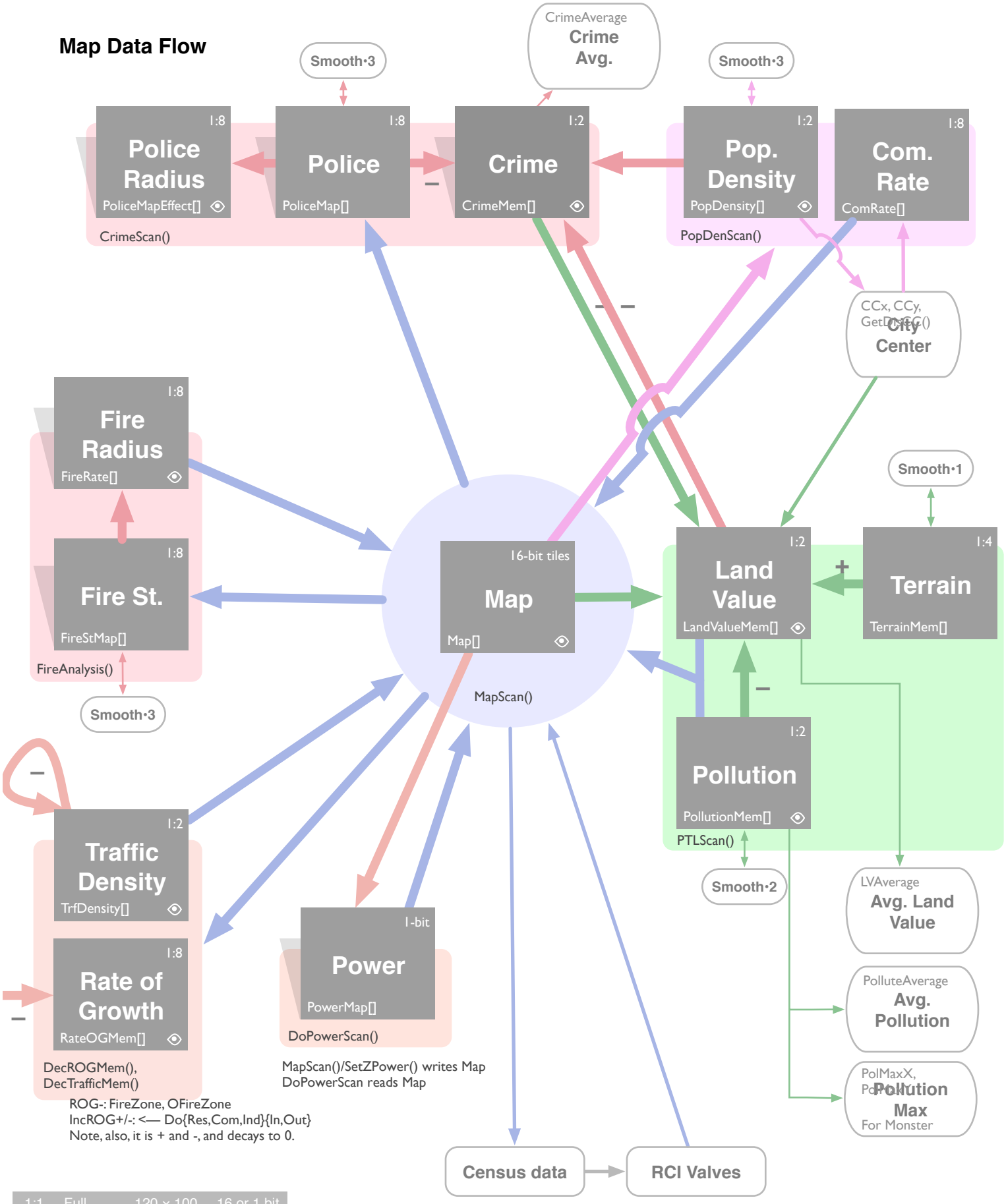


City simulation is broken down into 16 steps. Each revolution advances the city time by 1.

Every frame of the game, one of these 16 steps is performed. If the simulation speed is running at less than maximum, then on some frames no simulation work is done at all, and some steps are skipped entirely.

Arrange with circle at left center, and map scan at left edge. leaves more room for the right. OR flip vertically, and shove towards top, and have turn arrow be first on page.

Map Data Flow

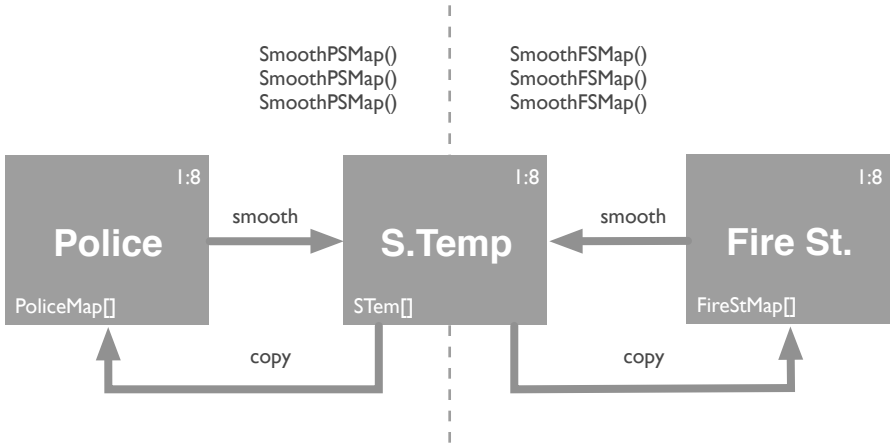


1:1	Full	120 × 100	16 or 1 bit
1:2	Half	60 × 50	8 bit
1:4	Quarter	30 × 25	8 bit
1:8	Small	15 × 13	16 bit

Map Smoothing

Smooth·3

Each of the three iterations of the police and fire coverage map blurring smoothes into STem[], then copies back.



Smooth·3

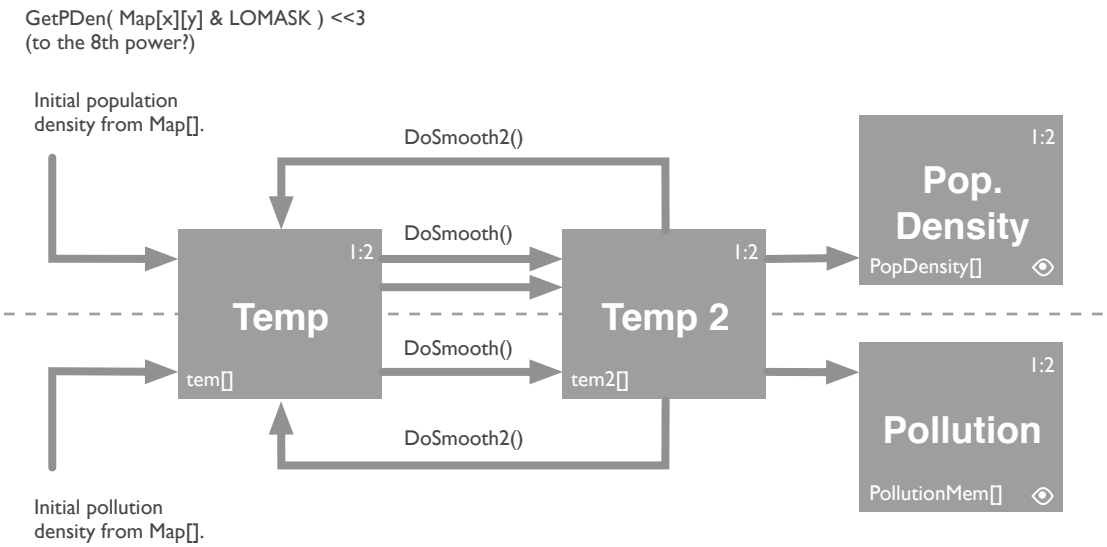
Population density is initially written into tem[], then smoothed three times using the two half scale buffers.

DoSmooth()
DoSmooth2()
DoSmooth()

Smooth·2

Pollution map is initially written into tem[], then smoothed twice using the two half scale buffers.

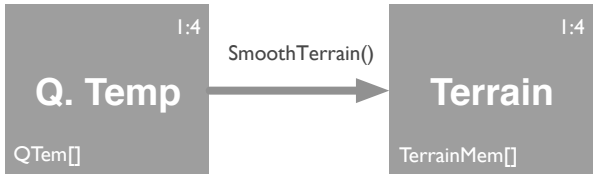
DoSmooth()
DoSmooth2()



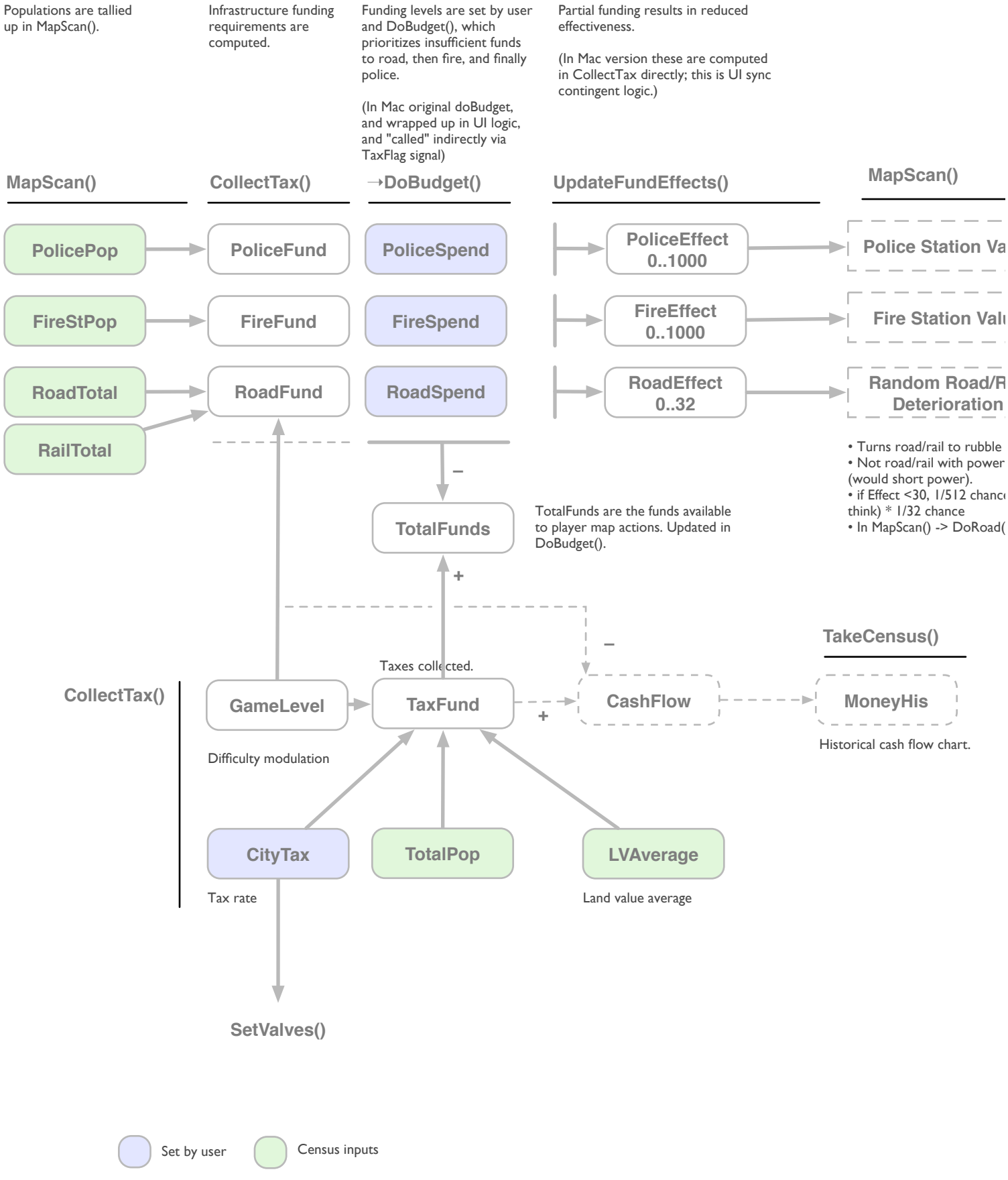
Smooth·2

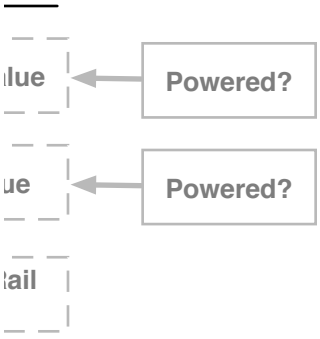
PTLScan() accumulates the natural landscape into QTem[], +15 for each tile < RUBBLE, which includes dirt, water, woods, and park.

It is smoothed once, into TerrainMem[], which will feed into the land value calculation.



Tax Effects


Populations are tallied up in MapScan().
Infrastructure funding requirements are computed.
Funding levels are set by user and DoBudget(), which prioritizes insufficient funds to road, then fire, and finally police.
(In Mac original doBudget, and wrapped up in UI logic, and "called" indirectly via TaxFlag signal)
Partial funding results in reduced effectiveness.
(In Mac version these are computed in CollectTax directly; this is UI sync contingent logic.)
MapScan()
CollectTax()
→DoBudget()
UpdateFundEffects()
MapScan()
PolicePop
FireStPop
RoadTotal
RailTotal
PoliceFund
FireFund
RoadFund
PoliceSpend
FireSpend
RoadSpend
PoliceEffect 0..1000
FireEffect 0..1000
RoadEffect 0..32
Police Station Va
Fire Station Val
Random Road/R Deterioration
• Turns road/rail to rubble
• Not road/rail with power (would short power).
• if Effect <30, 1/512 chanc think) * 1/32 chance
• In MapScan() -> DoRoad(
–
TotalFunds
TotalFunds are the funds available to player map actions. Updated in DoBudget().
+
–
TakeCensus()
Taxes collected.
CollectTax()
GameLevel
TaxFund
CashFlow
MoneyHis
+
Historical cash flow chart.
Difficulty modulation
CityTax
TotalPop
LVAverage
Tax rate
Land value average
SetValves()
Set by user
Census inputs



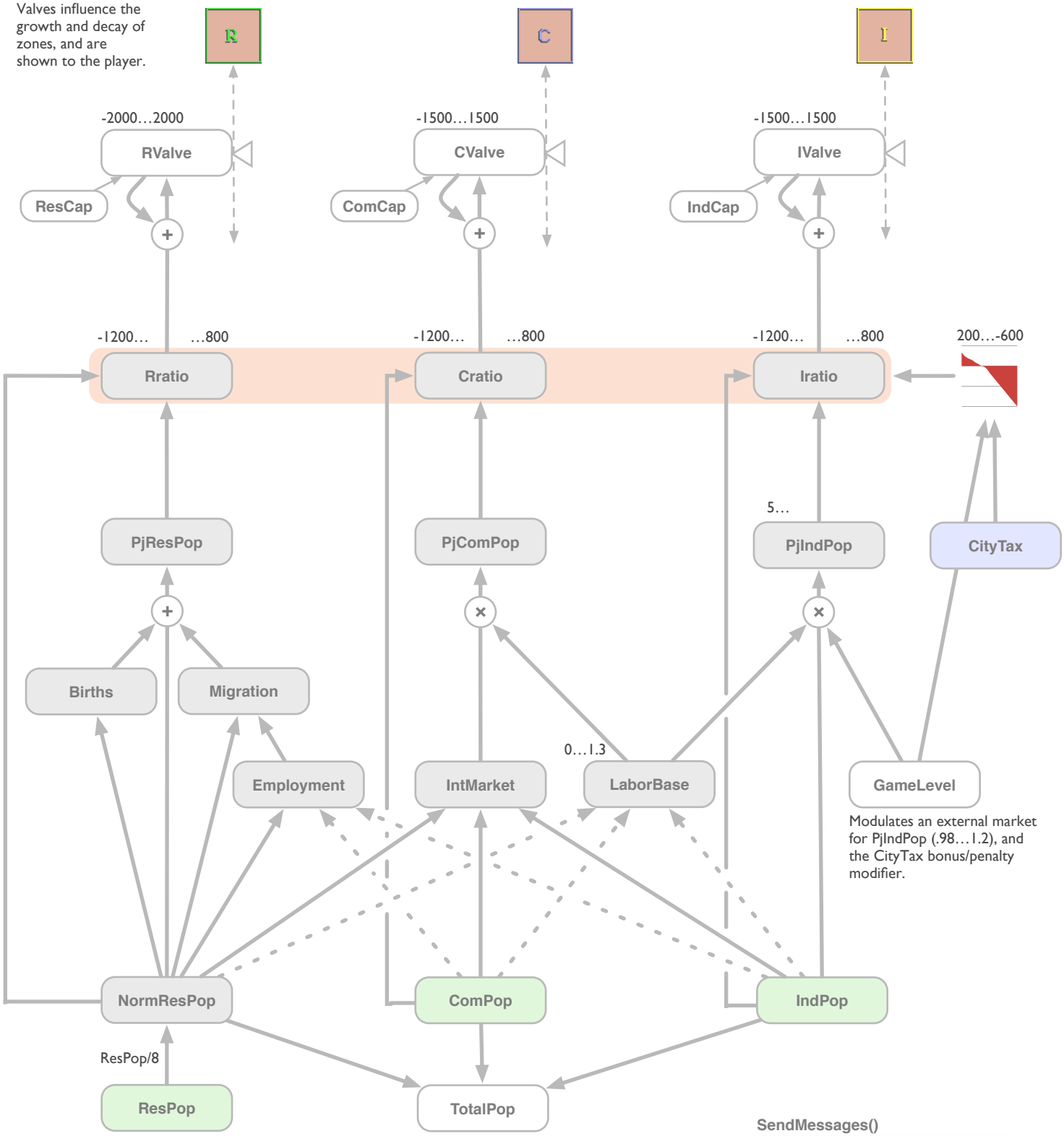
lines

e for rail (i

) DoRail()

SetValves()

Valves influence the growth and decay of zones, and are shown to the player.



Q: What is range of Res,Com,Ind-Pop? What do they count? show in MapScan() page?

Time delay of one census interval.

Global variable

Set by user

Census inputs

Temporary variables

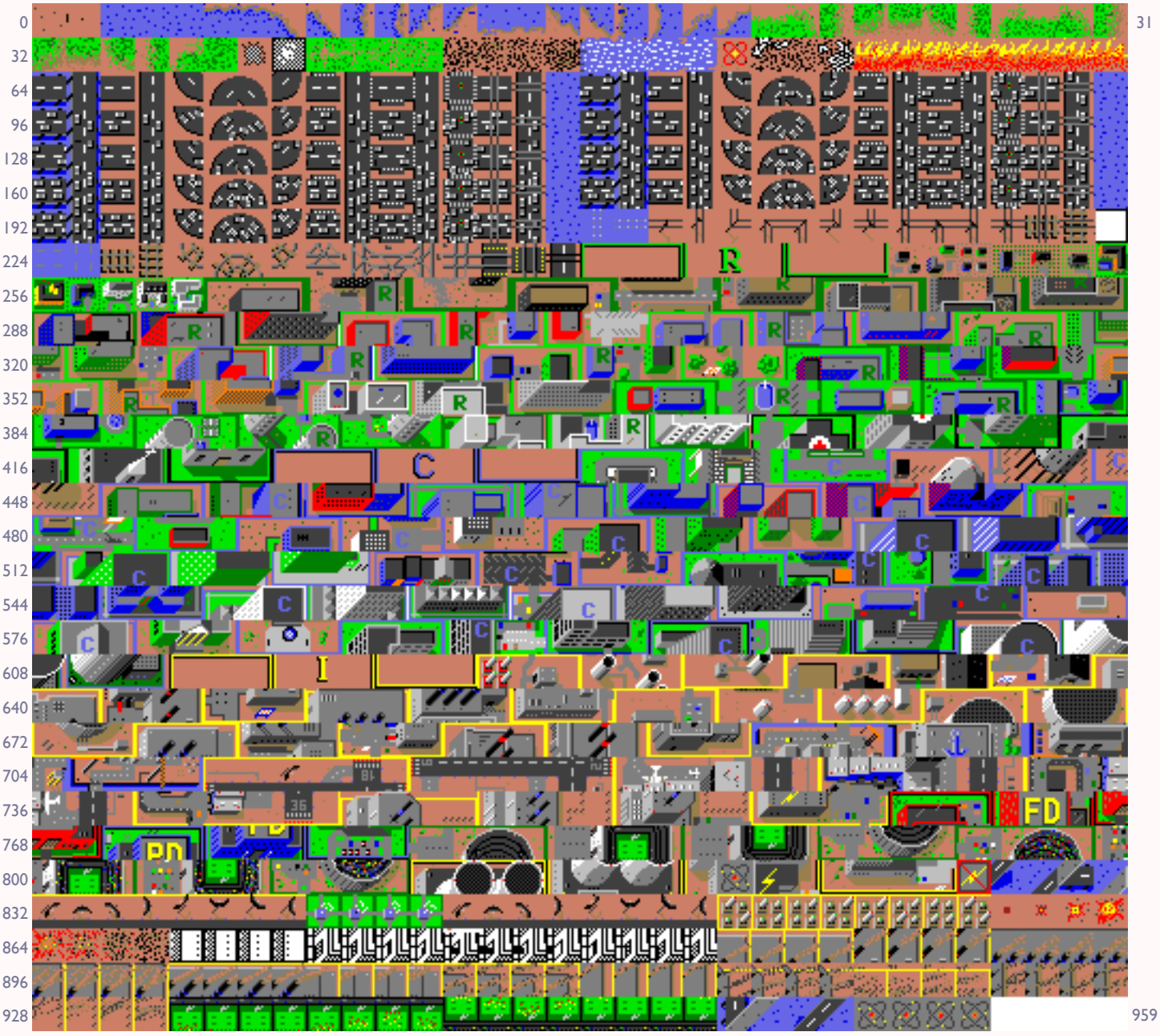
SendMessages()

Valves can be capped at zero by a lack of a desired structure:

ResCap	Stadium	ResPop	> 500
ComCap	Airport	ComPop	> 100
IndCap	Seaport	IndPop	> 70

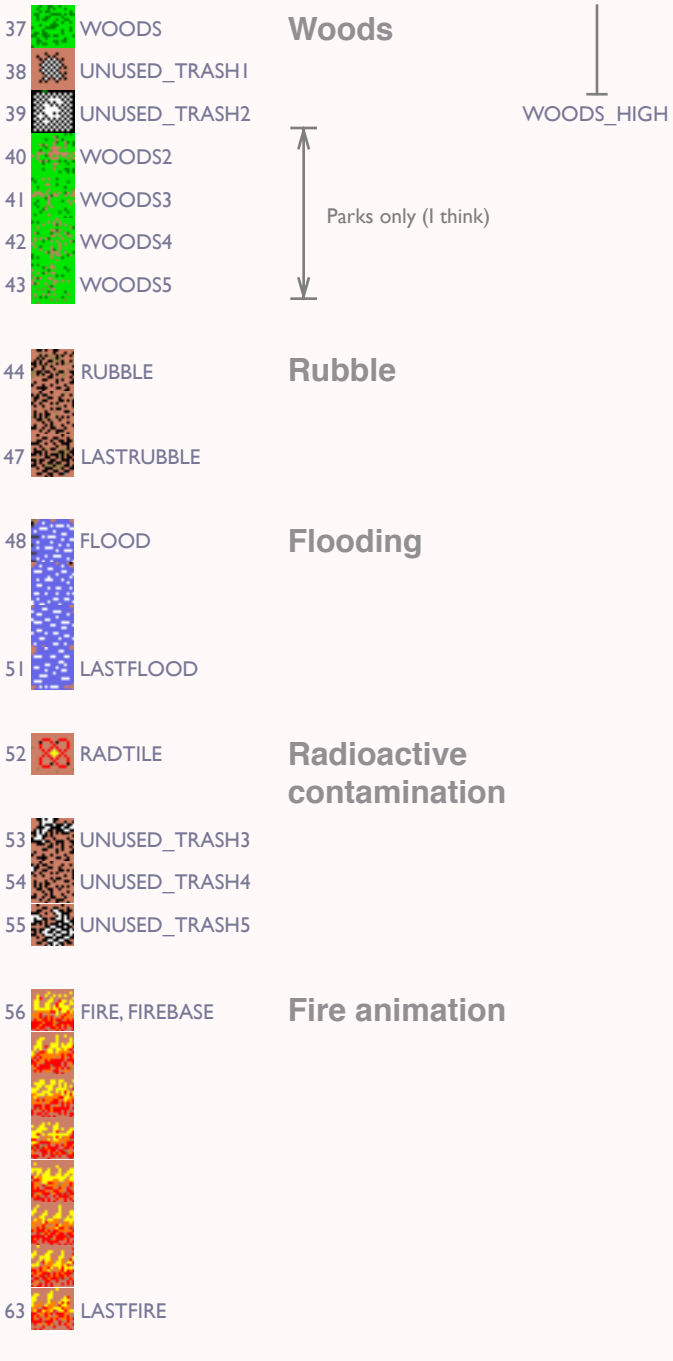
All Characters

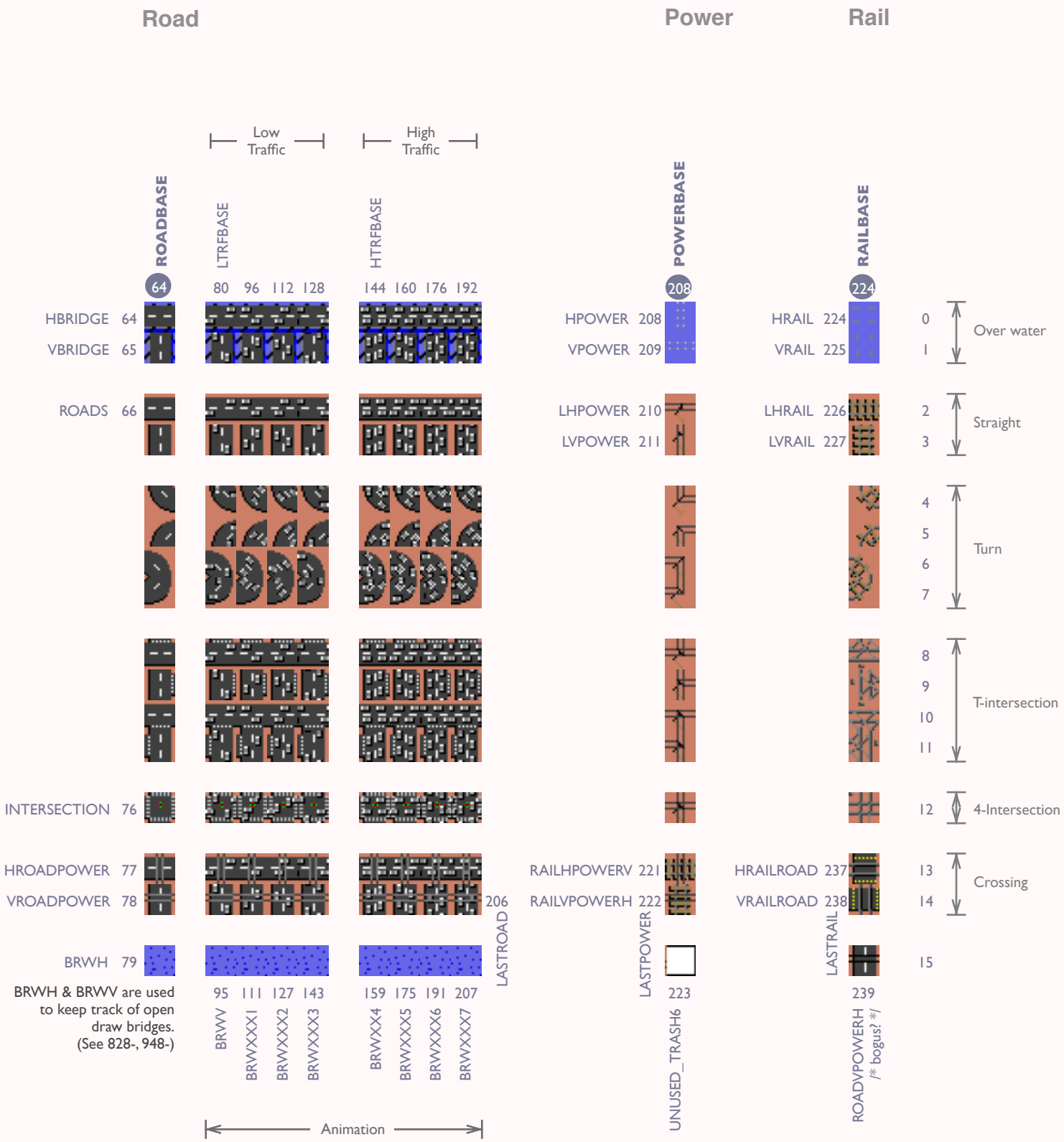
0-959



32x30 matrix of all 960 tiles; (column major form)


0 DIRT
Dirt
2 RIVER
3 REDGE
4 CHANNEL
Water
WATER_LOW
5 FIRSTRIVEDGE
Water/dirt border
+2xN for rotation
14 0 2
12 4
10 8 6
+1 for variation
15 1 3
13 5
11 9 7
20 LASTRIVEDGE
WATER_HIGH
21 TREEBASE
Woods/land border
WOODS_LOW
+N for rotation
7 0 1
6 2
5 4 3
+8 for variation
15 8 9
14 10
13 12 11
36 LASTTREE


37 WOODS
38 UNUSED_TRASH1
39 UNUSED_TRASH2
40 WOODS2
41 WOODS3
42 WOODS4
43 WOODS5
Woods
WOODS_HIGH
Parks only (I think)
44 RUBBLE
47 LASTRUBBLE
Rubble
48 FLOOD
51 LASTFLOOD
Flooding
52 RADTILE
Radioactive contamination
53 UNUSED_TRASH3
54 UNUSED_TRASH4
55 UNUSED_TRASH5
56 FIRE, FIREBASE
63 LASTFIRE
Fire animation



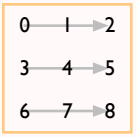
Patterns:
- &1 indicates if it is vertical.
- Why &15 for bridge logic?
- Tile = (Tile & 0x000F) + 64; // "NeutralizeRoad": turns any road tile into the first column. (w_con.c) (also masks off the status bits.



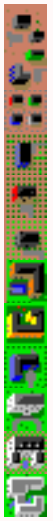
Residential Zone



An empty residential zone.



How a sequence of nine tiles are assembled by *SimCity* into a 3x3 building.



Houses

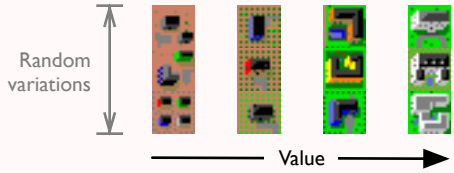
Single tile houses that indicate residential population increments from 1 to 8.

Low density residential zones fill in with these single tiles as population grows...



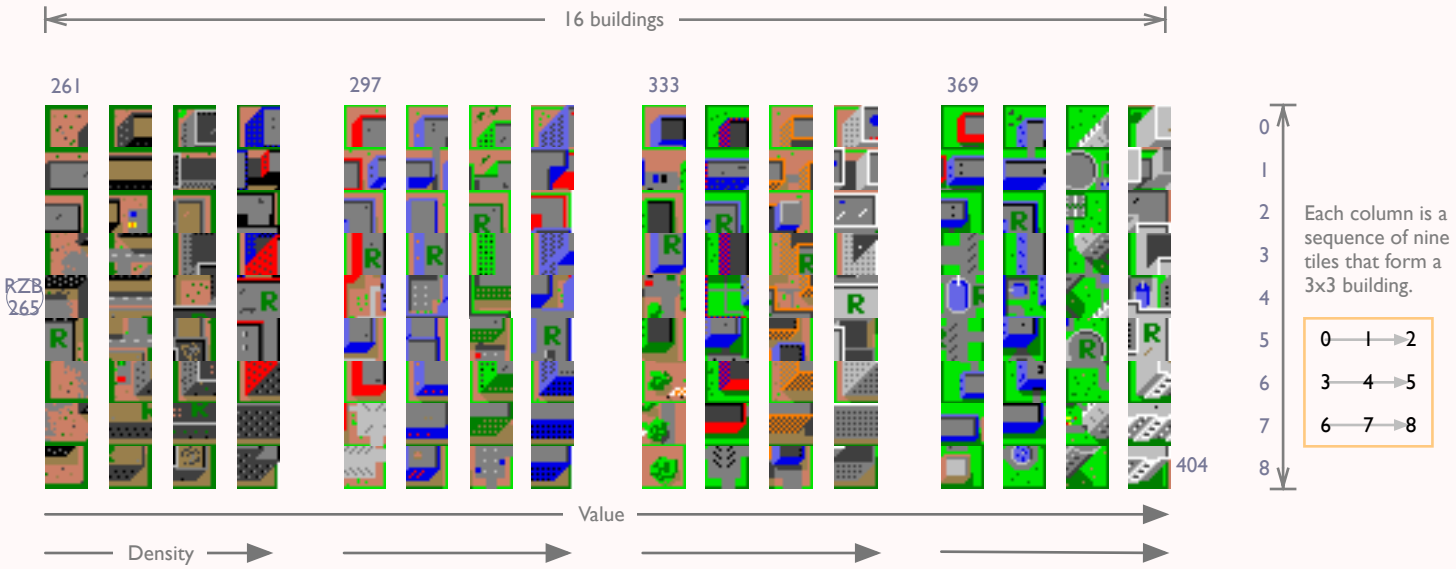
…eventually becoming residential buildings (described on the next page).

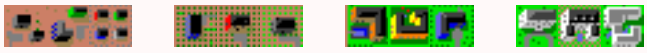
The tiles used correspond to land value.




```
s_zone.c / BuildHouse()
Map[xx][yy] = HOUSE + BLBNCNBIT + Rand(2) + (value * 3);
```

Residential Buildings





Value is also cued by a color gradient for houses and buildings.

16 buildings represent varying levels of value (0..3) and population (0..39) mapped to density (0..3).

Simple arithmetic translates value and density to the corresponding building tile sets.

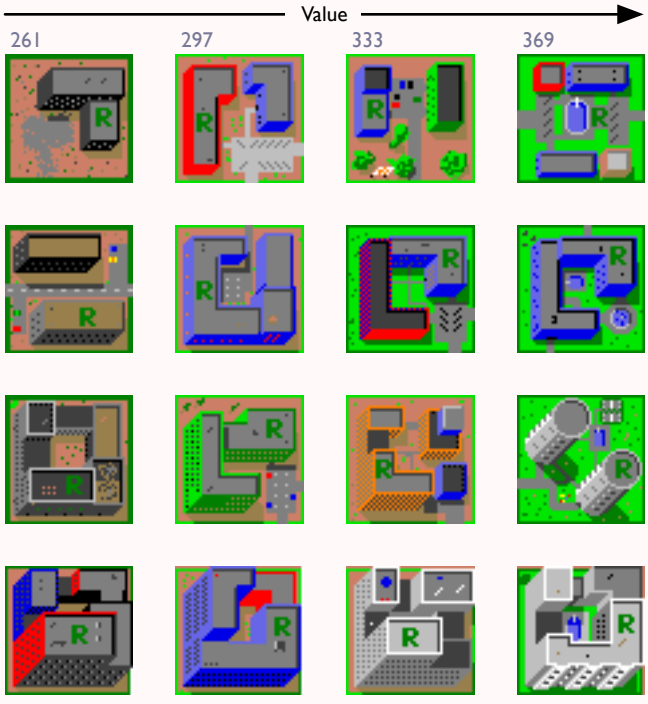
RZB +9 x[0...3] for density

+36x[0...3] for wealth

+[-4…+4] for tile

-4	-3	-2
-1	RZB	+1
+2	+3	+4

s_zone.c/ResPlop()
base = (((Value * 4) + Den) * 9) + RZB - 4;
ZonePlop(base);



Population
9 - 15
16 - 23
24 - 31
32 - 40

s_zone.c/DoResIn (for pop growth by 1?)

```
if (pop < 8) {

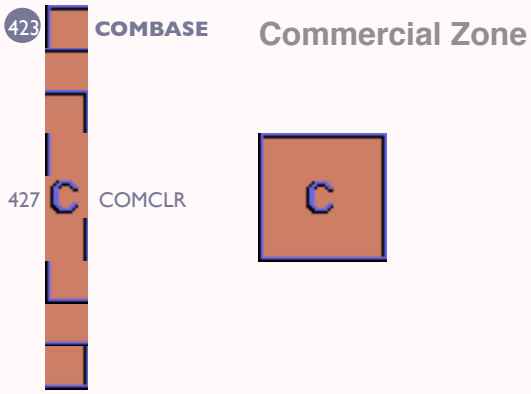
if (pop < 40) {
    ResPlop((pop / 8) - 1, value);
```

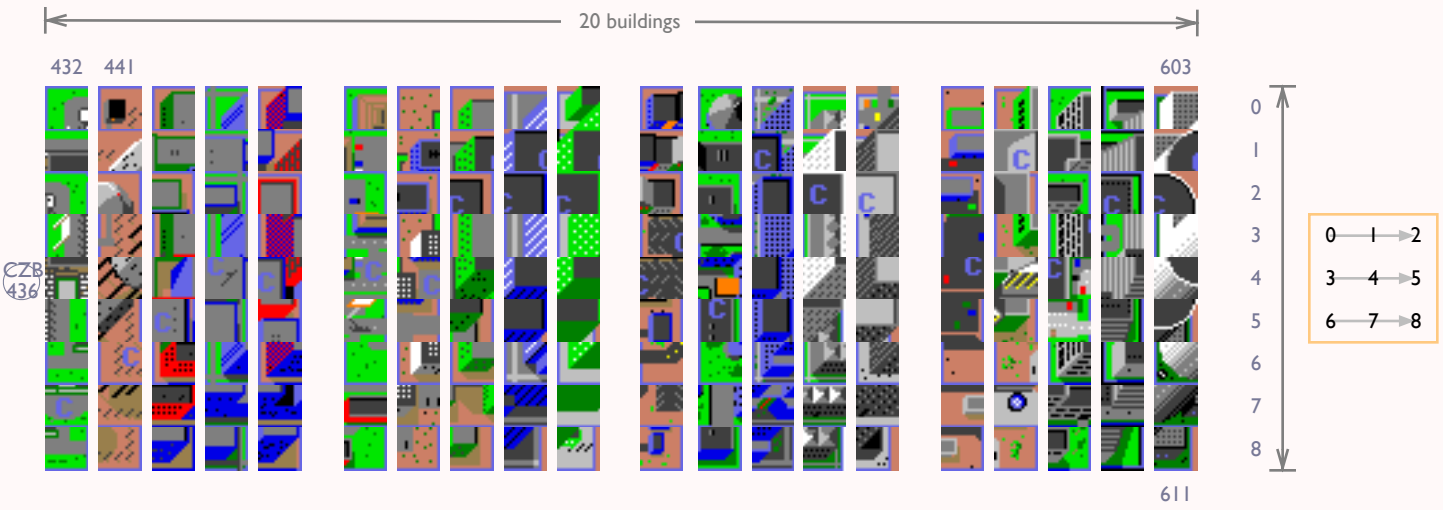
Hospitals and Churches

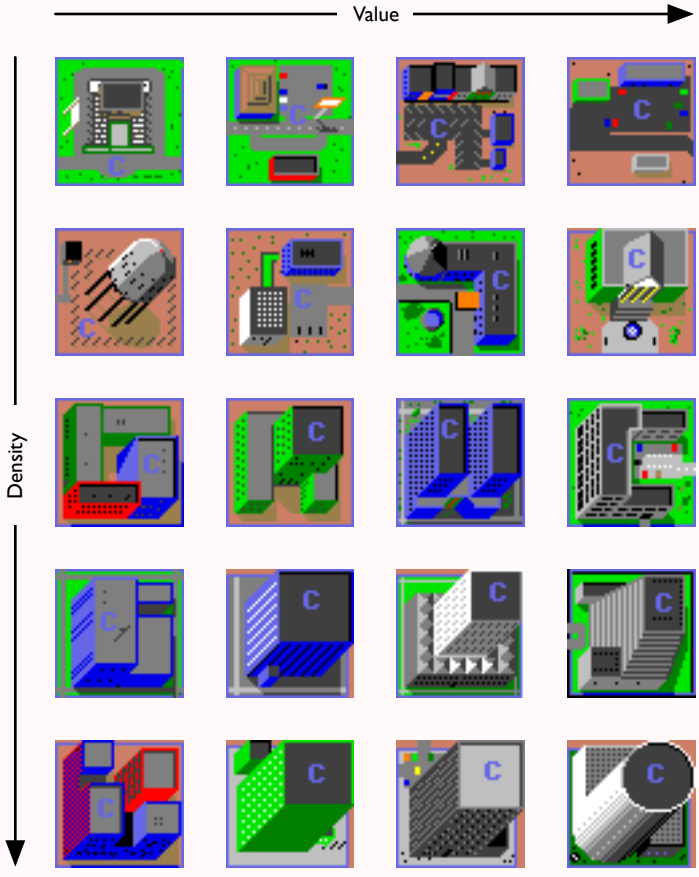




```
MakeHosp(void)
{
  if (NeedHosp > 0) {
    ZonePlop(HOSPITAL - 4);
    NeedHosp = FALSE;
    return;
  }
  if (NeedChurch > 0) {
    ZonePlop(CHURCH - 4);
    NeedChurch = FALSE;
    return;
  }
}
```







20 buildings represent varying levels of value (0..3) and density (0..4).

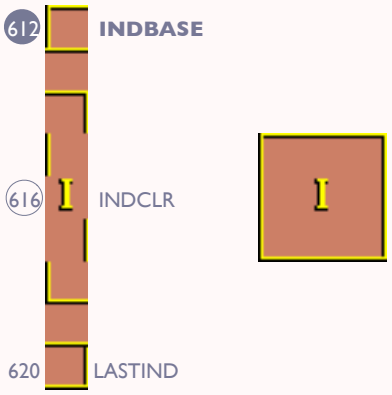
Simple arithmetic translates value and density to the corresponding building tile sets.

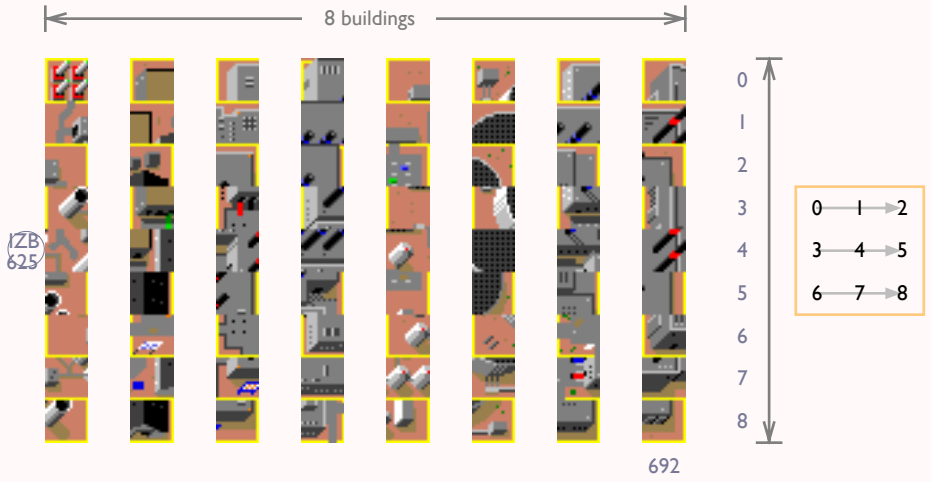
```
z = LandValueMem[SMapX >>1][SMapY >>1];
z = z >>5;
if (pop > z) return;

if (pop < 5) {
    ComPlop(pop, value);
    IncROG(8);
}
```

```
s_zone.c/ComPlop()
base = (((Value * 5) + Den) * 9) + CZB - 4;
ZonePlop(base);
```

Industrial Zone

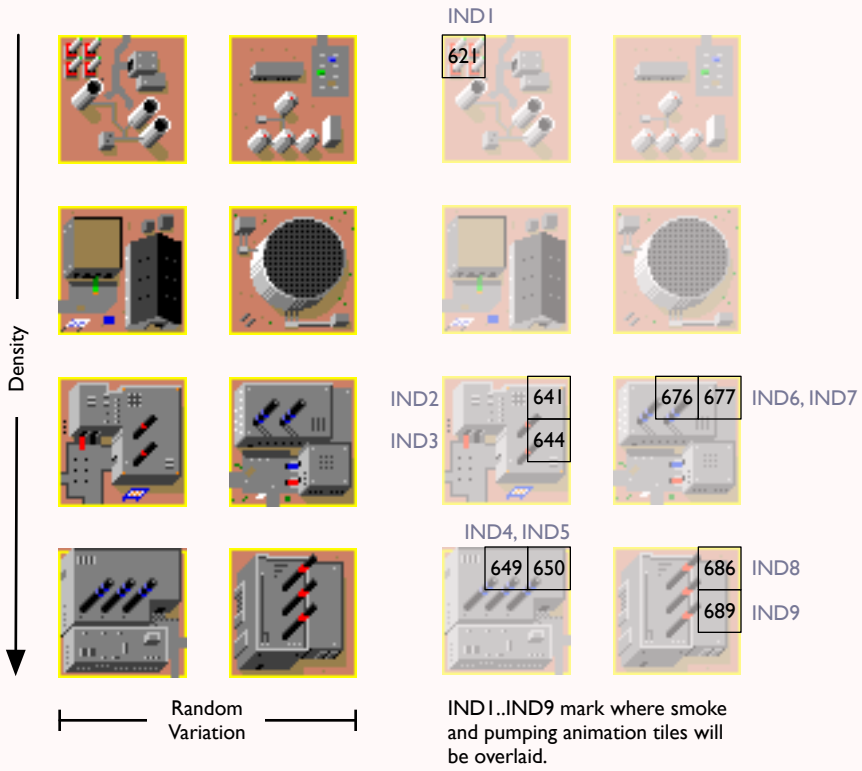




8 buildings represent varying densities (0..4), with 2 random variations.

Simple arithmetic translates value and density to the corresponding building tile sets.

base = (((Value * 4) + Den) * 9) + (IZB - 4);



IND1..IND9 mark where smoke and pumping animation tiles will be overlaid.

Seaport

693 PORTBASE

PORT 698

708 LASTPORT





4x4

```
    if (PwrOn) { /* post */
      if ((Map[SMapX + 1][SMapY - 1] & LOMASK) == RADAR)
            Map[SMapX + 1][SMapY - 1] = RADAR + ANIMBIT + CONDBIT + BURNBIT;
    } else
      Map[SMapX + 1][SMapY - 1] = RADAR + CONDBIT + BURNBIT;
—radar only spins if power is on.

/* comefrom: DoSPZone */
DoAirport(void)
{
  if (!(Rand(5))) {
    GeneratePlane(SMapX, SMapY);
    return;
  }
  if (!(Rand(12)))
    GeneratePlane(SMapX, SMapY);
}
```

Airport

709 AIRPORTBASE

AIRPORT 716

744



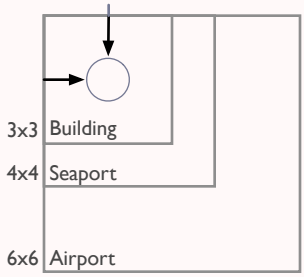
711 RADAR



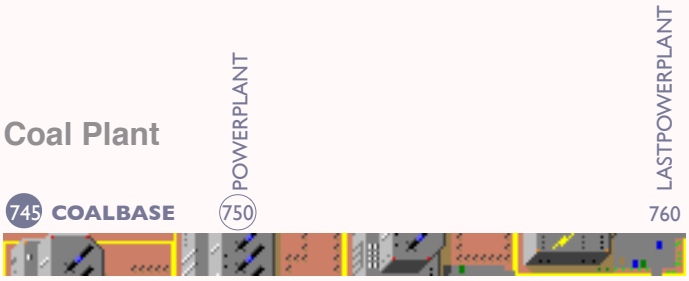
6x6

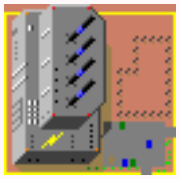
RADAR marks where the radar animation will be overlaid, which happens when the power is on.

All buildings locate the zone tile (i.e. PORT, AIRPORT, HOSPITAL) +1,+1 from the top left.



Coal Plant




4x4

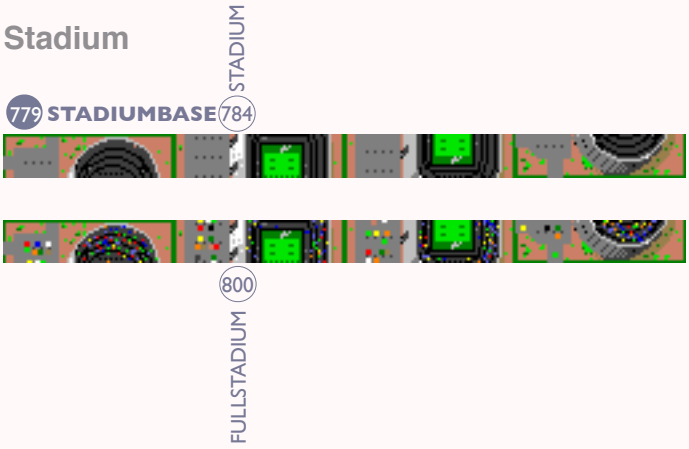
Fire & Police Station

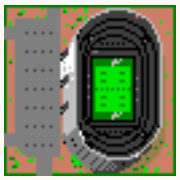


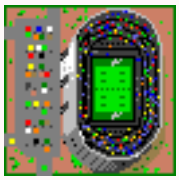




Stadium

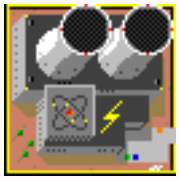


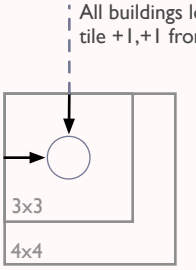

4x4



Nuclear Plant

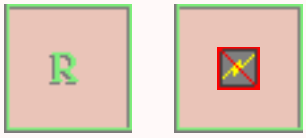



4x4



Power Outage

827  LIGHTNINGBOLT



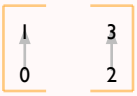
Blinking lightning bolt indicates a lack of power.

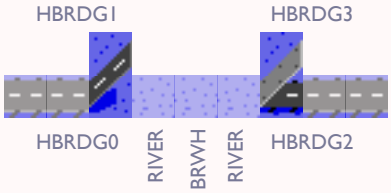
This blinking animation is not done through the tile character substitution technique, but is done while rendering the tiles to the screen.

Draw Bridge—Horizontal

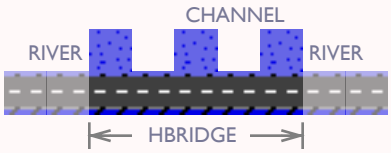
HBRDG0 … HBRDG3

828 





Bridges can open when boats are nearby, and close when they are far. BRWH bookmarks the open bridge.



s_sim.c/DoBridge() structures this logic as a toggle between two state configurations of a U shaped tile layout.

Airport Radar

832 

RADAR0 … RADAR7



Airport radar spins when the airport has power.

Park Fountain

840 

FOUNTAIN

Placed parks have a 1 in 5 chance of being an animating fountain, and a 4 in 5 chance of being WOODS.

Telecommunications

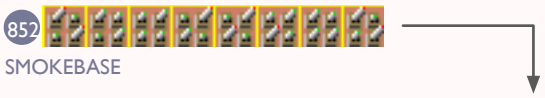
INDBASE2, TELEBASE 851 TELELAST

844 

s_traf.c/FindPTele(void) /* look for telecommunication on edges of zone */

Is this used? I see commented out code in OLPC Micropolis. Did Don add it? Is it just old unused code/assets?

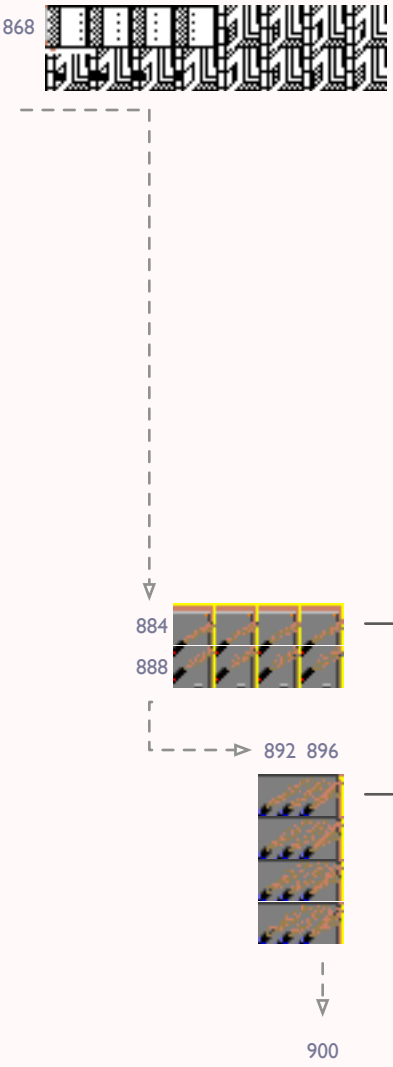
Industrial Activity



Tiny Explosion



Unused

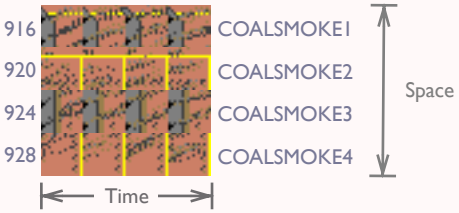


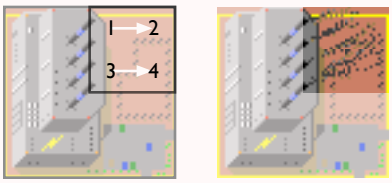
Smoke and pump animations slot into IND1…IND9 of industrial structures.

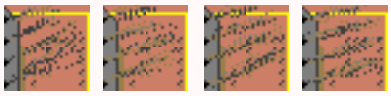
Note: Animation reward at high end (lots of activity) and a little at low end (an invitation, introductory reward).

Also, general green-ness as reward/value. Industrial always looks low value=un-green.

Coal Plant

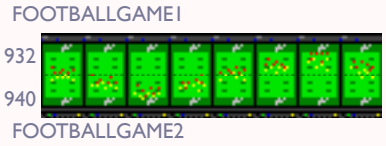


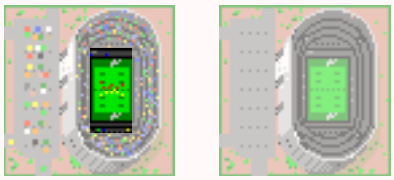




Smoke for coal power plant.

Football Game



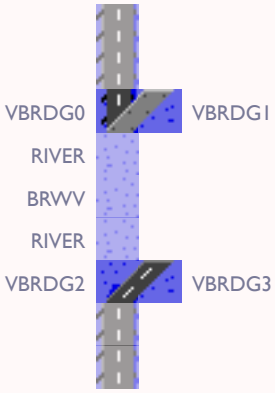


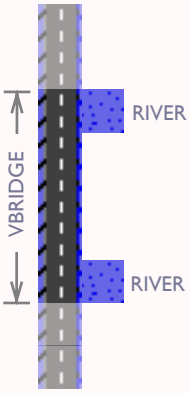
Animation for football game, organized into parallel top and bottom sequences.

The stadium also has empty and full states, whose tiles are stored adjacent (779-810).

Draw Bridge—Vertical







Vertical draw bridge logic works like the horizontal draw bridge (see 828-).

Nuclear Plant

